CO1 AT1

Kamesh.S 192521358

CSA-0412 Operating System

SHORT ANSWER TEST:

1.Suppose a modern Operating System supports multiple users and processes with shared resources like CPU, memory, and I/O devices.
(a) Classify different types of Operating Systems suitable for this scenario and justify your choice.
(b) Draw the structure of an Operating System showing how resources are managed.
(c) Write a sequence of system calls required to create and execute a new process in such an environment.

Modern Operating System:

A modern Operating System (OS) supports multiple users and processes while managing shared resources such as CPU, memory, and I/O devices.

Main Features:

1. Multi-User OS: Allows multiple users to access the system.
2. Multitasking OS: Runs multiple applications seemingly at the same time.
3. Multiprocessing OS: Uses multiple CPU cores or processors.
4. Time-Sharing OS: Shares CPU time among processes and users.
5. Resource Management: Allocates CPU, memory, and I/O devices efficiently.

Example: Linux and Windows are modern operating systems that support multitasking and resource management.

(a) Classify the different types of Operating Systems and justify your choice.

A Modern Operating System (OS) manages multiple users, processes, and shared resources such as CPU, memory, and I/O devices.

Types of Operating Systems

1. Multi-User Operating System

- Allows multiple users to access the computer system simultaneously.
- Provides separate user accounts, permissions, and resource access.
- Example: UNIX, Linux.

2. Multitasking Operating System

- Allows multiple applications to run seemingly at the same time.
- The CPU switches between processes to provide efficient execution.
- Example: Windows 11, Linux.

3. Multiprocessing Operating System

- Uses multiple CPU cores or processors to execute processes efficiently.
- Improves processing performance and supports parallel execution.
- Example: Linux, Windows.

4. Time-Sharing Operating System

- Allocates CPU time to multiple users and processes.
- Provides quick responses and fair CPU scheduling.
- Example: UNIX.

5. Network Operating System

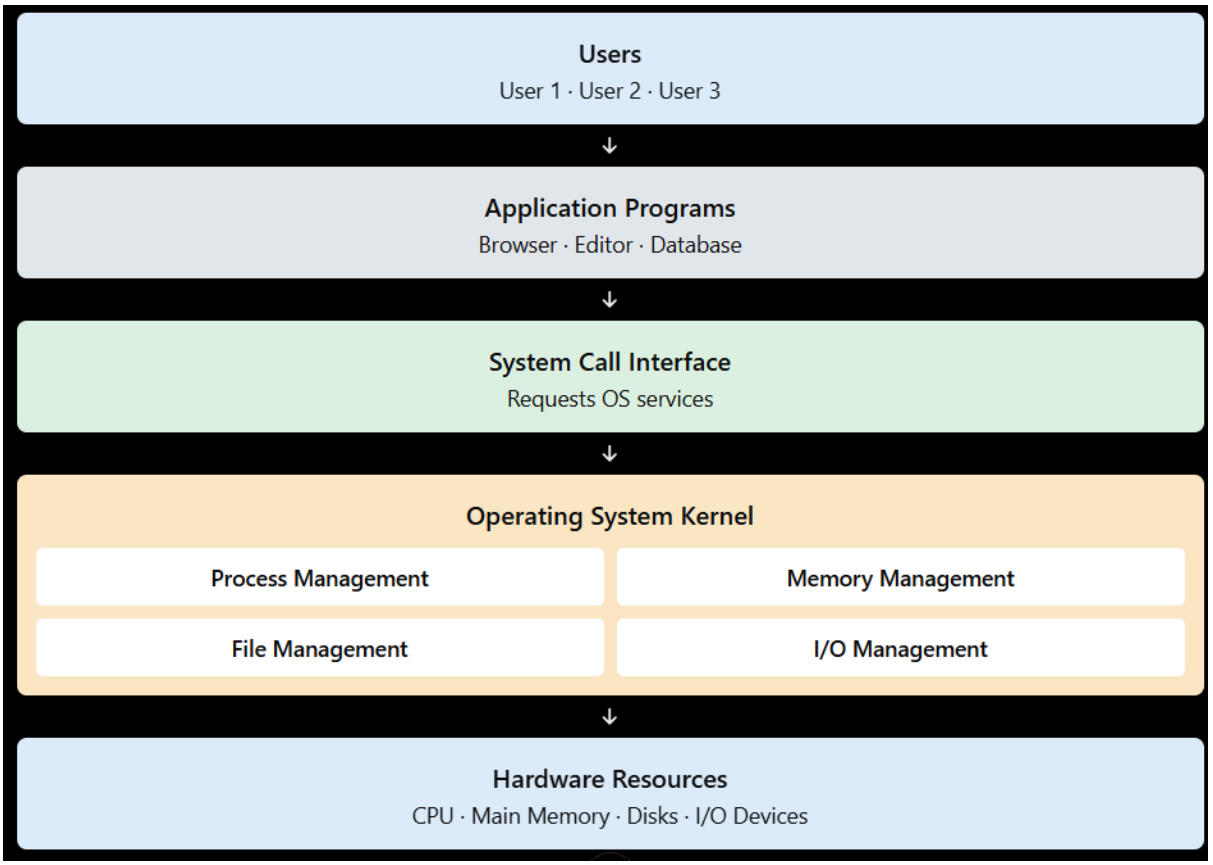
- Manages computers and resources connected through a network.
- Supports shared files, printers, and other network services.
- Example: Windows Server.

Justification:

A combination of multi-user, multitasking, multiprocessing, and time-sharing features is suitable for this scenario because it supports multiple users, executes multiple processes, shares CPU time, and manages hardware resources efficiently.

(b) Draw the structure of an Operating System showing resource management:

Operating System Structure:



Explanation of the Structure

1. Users: Access the system through user accounts.
2. Applications: Programs that request OS services.
3. System Call Interface: Transfers requests from applications to the kernel.
4. Process Management: Schedules processes and allocates CPU time.
5. Memory Management: Allocates and protects memory.
6. File Management: Organizes files and storage.
7. I/O Management: Controls input and output devices.
8. Hardware: Provides the physical resources used by programs

(c) Write a sequence of system calls to create and execute a new process.

In a UNIX/Linux operating system, the `fork()`, `exec()`, and `wait()` system calls are commonly used for process creation and execution.

Sequence of System Calls

Step 1: `fork()`

- Creates a new child process.
- The child process is initially a copy of the parent process.

Step 2: `exec()`

- Replaces the child process's program with a new program.
- The new program begins execution if the call succeeds.

Step 3: `wait()`

- Allows the parent to wait for the child process to finish.
- Helps the parent collect the child's termination status.

Step 4: `exit()`

- Terminates the child process after execution is complete.

C Program Example:

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

#include <stdlib.h>

int main()

{

   pid_t pid = fork();
```

```
    if (pid == 0)
    {
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec failed");
        exit(1);
    }
    else if (pid > 0)
    {
        wait(NULL);
        printf("Child process completed\n");
    }
    else
    {
        perror("fork failed");
    }
     return 0;
}
```

Output:

Child process completed

2.Consider an Operating System managing multiple files and devices for concurrent users.

(a) Categorize system calls used for file and device management.

(b) Illustrate how the OS handles file operations using a layered structure diagram.

(c) Write a pseudo-code or C-based system call sequence to open, read, and close a file.

Operating System Managing Multiple Files and Devices

An Operating System (OS) manages multiple files, hardware devices, and concurrent users. It provides system calls to access files and control devices safely and efficiently.

Main functions:

1. File Management: Creates, opens, reads, writes, and closes files.
2. Device Management: Controls devices such as printers, keyboards, disks, and monitors.
3. Resource Allocation: Allocates CPU time, memory, and I/O devices to different processes.
4. Concurrent Access: Coordinates multiple users and processes accessing shared resources.
5. Protection and Security: Checks permissions to prevent unauthorized file and device access.

Example: In Linux, multiple users can work simultaneously while the OS manages their files, processes, and device requests.

(a) Categorize System Calls Used for File and Device Management

System calls are the interfaces through which user programs request services from the operating system.

1. File Management System Calls

These system calls are used to create, open, read, write, and close files.

- Create: Creates a new file. Example: `creat()`

- Open: Opens an existing file. Example: `open()`
- Read: Reads data from a file. Example: `read()`
- Write: Writes data into a file. Example: `write()`
- Close: Closes an opened file. Example: `close()`
- Delete: Removes a file. Example: `unlink()`

2. Device Management System Calls

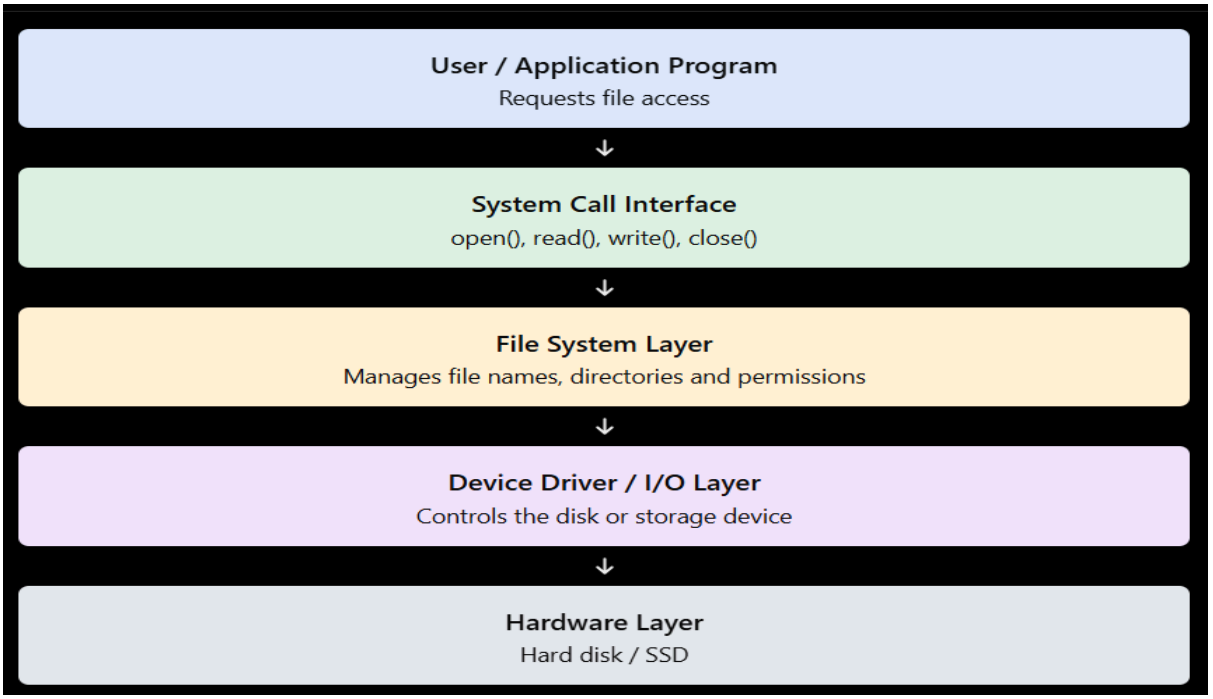
These system calls manage and control hardware devices such as keyboards, printers, disks, and other I/O devices.

- Request Device: Requests access to a device.
- Release Device: Releases a device after use.
- Read/Write Device: Transfers data between a device and a program.
- Get Device Information: Retrieves information about a device.
- Control Device: Changes device settings using calls such as `ioctl()`.

Example: When a program reads data from a disk, the OS uses file and device management services to access the required data.

(b) Layered Structure Diagram for File Operations

The operating system uses a layered structure to process file operations. Each layer performs a specific task and communicates with the layer above



Working of the Layered Structure

1. The application requests a file operation.
2. The system call interface receives the request.
3. The file system checks the file name, permissions, and file information.
4. The device driver communicates with the storage device.
5. The hardware performs the required read or write operation.
6. The result is returned to the application through the OS layers.

Advantages: This structure provides modularity, easier maintenance, better security, and simpler device management.

(c) C Program to Open, Read, and Close a File

The following C program uses system calls to open a file, read its contents, display them, and close the file. It is suitable for Linux or a Unix-like operating system.

Program

```
#include <stdio.h>

#include <fcntl.h>

#include <unistd.h>

#include <stdlib.h>

int main()

{

   int fd;

   char buffer[100];

   ssize_t n;



   fd = open("sample.txt", O_RDONLY);

   if (fd == -1)
```

```
    {
        perror("File opening failed");
        return 1;
    }
    n = read(fd, buffer, sizeof(buffer) - 1);
    if (n == -1)
    {
        perror("File reading failed");
        close(fd);
        return 1;
    }
    buffer[n] = '\0';
    printf("File contents:\n%s\n", buffer);
    close(fd);
    return 0;
}
```

Pseudocode

1. START
2. Declare a file descriptor and a character buffer.
3. Open `sample.txt` in read-only mode.
4. If opening fails, display an error and stop.
5. Read the file contents into the buffer.
6. If reading fails, display an error and close the file.
7. Add the string terminator and display the contents.
8. Close the file using `close()`.
9. STOP.

Explanation of System Calls

System call	Purpose
`open()`	Opens the file in read-only mode.
`read()`	Reads data from the opened file.
`close()`	Releases the file descriptor and closes the file.

3.Assume a real-time Operating System used in an embedded system controlling industrial machinery.

(a) Identify the type of OS and explain why it is suitable for real-time applications.

(b) Draw a diagram representing the interaction between hardware, kernel, and user processes.

(c) List and explain system calls required for process synchronization in this system.

Real-Time Operating System (RTOS)

A Real-Time Operating System (RTOS) is an operating system designed to execute tasks and respond to events within a specified time limit. It is used in embedded systems to control industrial machinery, robots, and automated equipment.

Main Features

1. Fast Response: Responds quickly to sensor inputs and control signals.
2. Predictable Execution: Performs important tasks within defined time limits.
3. Priority Scheduling: Executes critical tasks before lower-priority tasks.
4. Resource Management: Manages CPU, memory, and I/O devices efficiently.
5. Synchronization: Coordinates multiple tasks that share resources.
6. Reliability: Supports continuous and dependable industrial operation.

Example: In an automated factory, an RTOS monitors temperature sensors and controls motors according to the sensor readings within the required time.

(a) Identify the Type of OS and Explain Why It Is Suitable

Type of OS: Real-Time Operating System (RTOS)

A Real-Time Operating System is an operating system that processes events and produces responses within a specified time limit. It is used in embedded systems that control industrial machinery, robots, medical equipment, and automated systems.

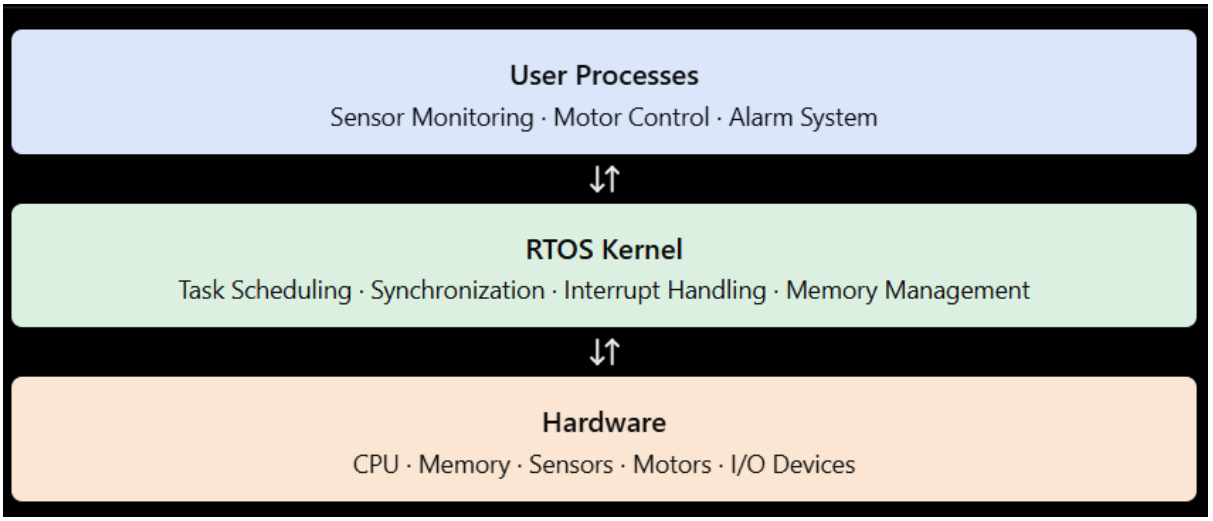
Why RTOS Is Suitable

1. Fast Response: Responds quickly to sensor inputs and control signals.
2. Deterministic Behaviour: Performs critical tasks within predictable time limits.
3. Priority-Based Scheduling: Gives higher priority to important tasks, such as emergency shutdown.
4. Efficient Resource Management: Manages CPU, memory, and I/O devices efficiently.
5. Reliability: Supports continuous operation in industrial environments.
6. Task Synchronization: Coordinates multiple tasks that share resources.

Example: In a factory, an RTOS can process temperature sensor readings and send control signals to machinery within a required time limit.

RTOS types include Hard Real-Time OS and Soft Real-Time OS. A hard RTOS is appropriate when missing a deadline could cause serious failure in a critical control system.

(b) Diagram Showing Interaction Between Hardware, Kernel, and User Processes



Explanation

1. User Processes: Perform tasks such as monitoring sensors and controlling motors.
2. RTOS Kernel: Schedules tasks, handles interrupts, and coordinates shared resources.
3. Hardware: Includes the processor, memory, sensors, and industrial control devices.
4. Communication: User processes request kernel services, while the kernel interacts with hardware through device drivers and interrupt mechanisms.

(c) System Calls Required for Process Synchronization

System calls allow processes to coordinate their activities and safely access shared resources.

1. Semaphore Operations

- `sem_wait()` – Waits for a semaphore and may block the task until the resource is available.
- `sem_post()` – Signals the semaphore, allowing another waiting task to proceed.

Use: Coordinates access to shared data or equipment.

2. Mutex Operations

- `mutex_lock()` – Locks a shared resource so that only one task can access it at a time.
- `mutex_unlock()` – Releases the lock after the task finishes using the resource.

Use: Prevents multiple tasks from modifying shared data simultaneously.

3. Condition Variable Operations

- `cond_wait()` – Suspends a task until a specified condition is signalled.
- `cond_signal()` – Wakes a task waiting for that condition.

Use: Allows a control task to wait until sensor data becomes available.

4. Message Queue Operations

- `send()` – Sends a message from one task to another.
- `receive()` – Receives a message, optionally waiting until one becomes available.

Use: Transfers sensor readings or machine status between tasks.

5. Event and Task Synchronization

- `event_wait()` – Waits for a specific event.
- `event_signal()` – Signals that an event has occurred.

4.A distributed computing system consists of multiple interconnected computers sharing resources over a network.

(a) Classify the type of Operating System used and compare it with centralized OS.

(b) Draw the architecture of a distributed OS showing communication between nodes.

(c) Write the sequence of system calls involved in inter-process communication (IPC).

Distributed Computing System

A Distributed Computing System consists of multiple interconnected computers that communicate and share resources over a network. Each computer, called a node, has its own processor and memory and works together with other nodes to complete tasks.

Main Features

1. Resource Sharing: Computers share files, storage, processing power, and other resources.
2. Communication: Nodes exchange data through a network.
3. Load Distribution: Tasks are distributed among computers to improve performance.
4. Scalability: New computers can be added to increase system capacity.
5. Fault Tolerance: The system may continue operating when a node fails, depending on its design.

Example: A distributed system can use several computers to process large amounts of data simultaneously.

(a) Classify the OS and Compare It with a Centralized OS

Distributed Operating System (DOS): A distributed OS manages multiple interconnected computers and coordinates their resources so that they can work together as a unified system.

Features of a Distributed OS

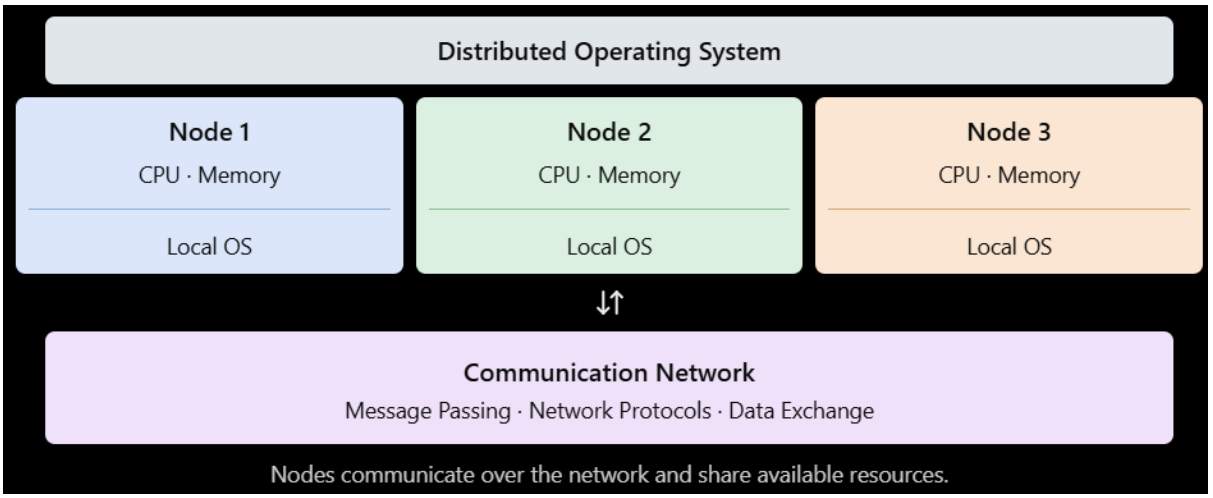
1. Resource Sharing: Shares files, storage, processing power, and devices across multiple computers.
2. Communication: Enables computers to exchange data through a network.
3. Load Sharing: Distributes tasks among computers to improve performance.
4. Reliability: Other nodes may continue working if one node fails, depending on the system design.
5. Scalability: Allows additional computers to be added to the system.

Comparison: Distributed OS vs Centralized OS

Distributed OS	Centralized OS
Uses multiple interconnected computers.	Primarily relies on one central computer.
Resources are distributed across nodes.	Resources are managed by the central system.
Processing can occur on different nodes.	Processing is mainly performed by the central system.
Network communication is essential.	Other terminals may connect to the central computer.
Can offer improved fault tolerance with proper design.	Failure of the central system can affect the entire service.
Example: Amoeba.	Example: A traditional single-computer system running UNIX.

(b) Architecture of a Distributed OS

The following diagram illustrates communication between nodes in a distributed system.



Explanation

1. Nodes: Each node is a computer with its own CPU, memory, and local operating system components.
2. Communication Network: Connects the computers and transfers messages between them.
3. Distributed OS Services: Coordinate resource sharing, task distribution, and communication.
4. Resource Sharing: A node can request files or services from another node.
5. Coordination: Nodes exchange information to perform distributed tasks.

(c) Sequence of System Calls Involved in IPC

Inter-Process Communication (IPC) allows processes to exchange data and coordinate their activities.

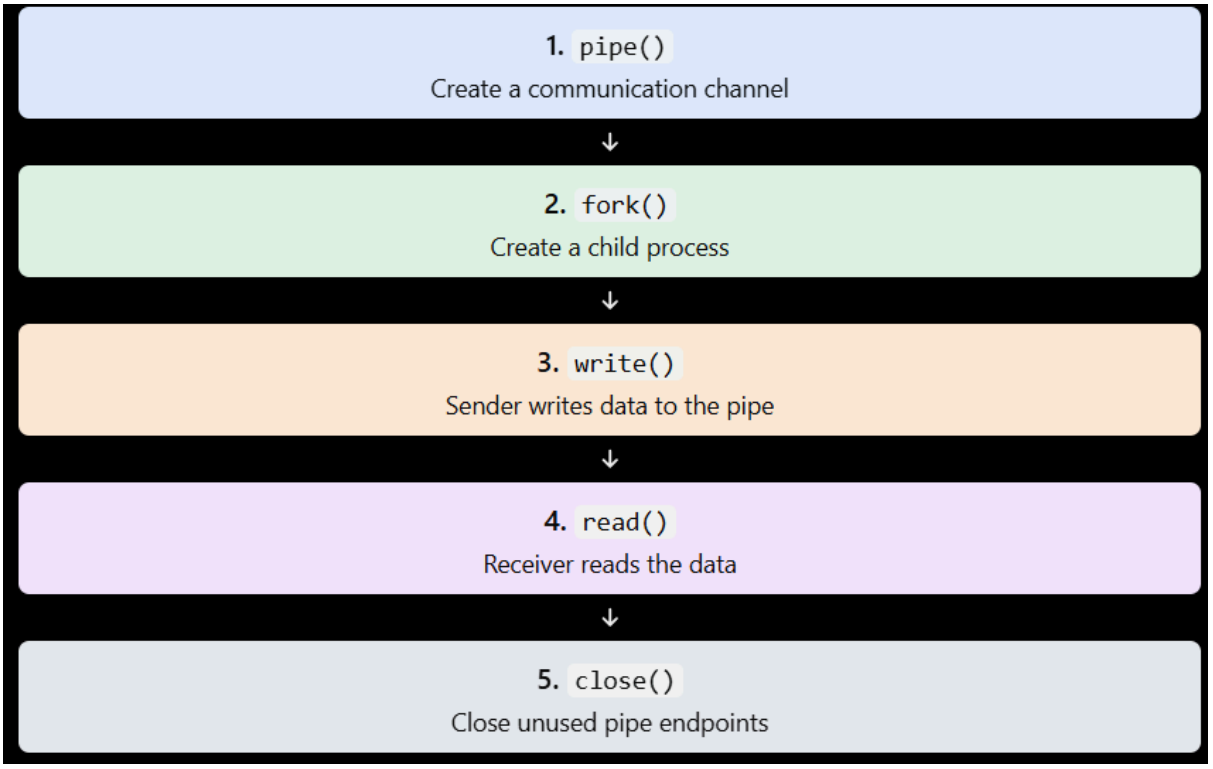
Common IPC System Calls

- `pipe()` – Creates a communication channel between processes.
- `fork()` – Creates a child process that can communicate with its parent.
- `write()` – Sends data through a communication channel.

- `read()` – Receives data from a communication channel.
- `close()` – Closes an unused communication endpoint.
- `socket()` – Creates a network communication endpoint.
- `connect()` – Connects a client socket to a server.
- `send()` / `recv()` – Transmit and receive data through sockets.

Sequence of IPC System Calls

For communication between two related processes using a pipe:



Simple Pseudocode:

START

Create pipe using pipe()

Create child process using fork()

IF parent process

Close read endpoint

Send message using write()

Close write endpoint

ELSE

Close write endpoint

Receive message using read()

Display received message

Close read endpoint

END IF

STOP

5.Consider a scenario where an Operating System must manage process scheduling, memory allocation, and device communication efficiently.

(a) Explain the major functions of an OS in handling these tasks.

(b) Draw a block diagram representing OS components and their interactions.

(c) Write system call examples for memory allocation and device Management.

Operating System Resource Management

An Operating System (OS) manages process scheduling, memory allocation, and device communication efficiently. It acts as an interface between users, applications, and computer hardware.

Major Functions

1. Process Scheduling: The OS allocates CPU time to different processes and decides which process executes next.

2. Memory Allocation: The OS allocates memory to programs, tracks memory usage, and releases memory when it is no longer needed.

3. Device Communication: The OS uses device drivers and I/O operations to communicate with hardware devices such as keyboards, printers, and disks.

4. Resource Management: The OS coordinates CPU, memory, and I/O devices to improve system performance.

5. Security and Protection: The OS prevents unauthorized access to memory, files, and devices.

Example:

When a user opens a web browser, the OS allocates memory, schedules the browser process, and manages communication with input/output devices.

(a) Major Functions of an Operating System

An Operating System (OS) is system software that manages computer hardware and software resources. It controls process scheduling, memory allocation, and communication with input/output devices.

1. Process Management

- Creates, schedules, and terminates processes.
- Allocates CPU time to processes using scheduling algorithms such as FCFS, SJF, and Round Robin.
- Supports process synchronization and coordination.

2. Memory Management

- Allocates memory to programs when required and releases it after use.
- Tracks memory usage and prevents processes from accessing unauthorized memory.
- Supports virtual memory and efficient memory utilization.

3. Device Management

- Controls devices such as keyboards, printers, disks, and monitors.
- Uses device drivers to communicate with hardware.

- Manages device allocation, buffering, and I/O operations.

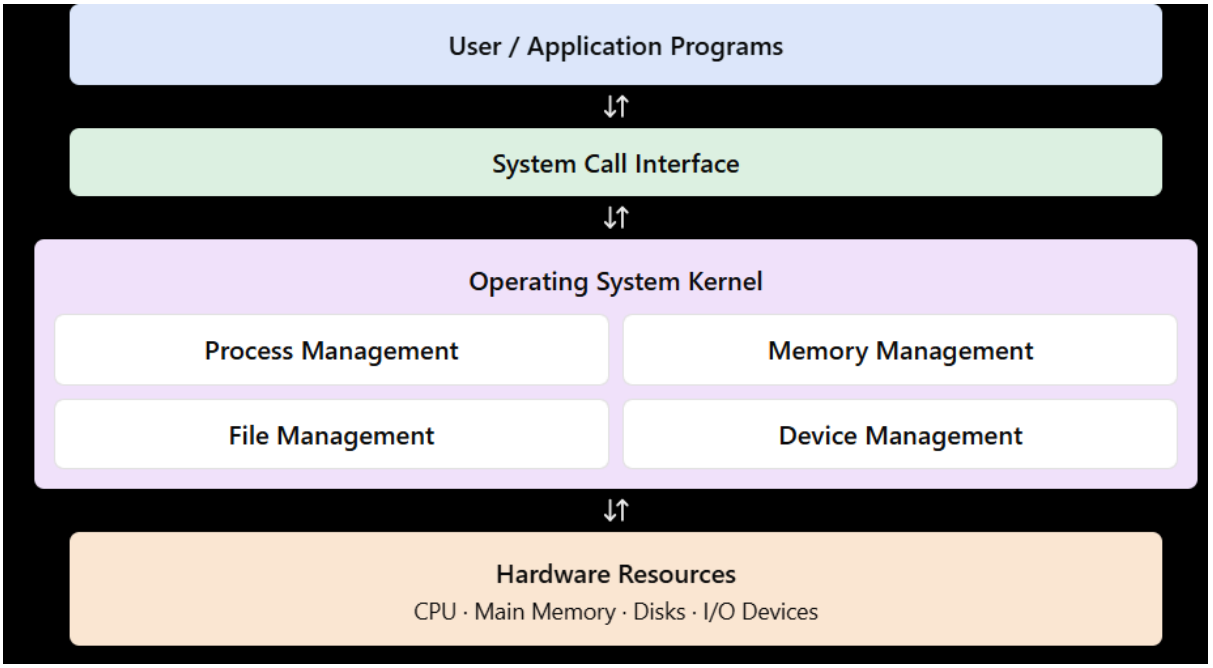
4. File Management

- Creates, reads, writes, and deletes files.
- Organizes files into directories.
- Controls file permissions and access.

5. Security and Protection

- Authenticates users and controls access to system resources.
- Protects data and processes from unauthorized access.

(b) Block Diagram of OS Components and Their Interactions



Explanation

1. User/Application Programs: Request OS services to execute tasks.
2. System Call Interface: Provides access to OS functions through system calls.
3. Kernel: Coordinates process scheduling, memory allocation, file operations, and device communication.
4. Hardware: Executes instructions, stores data, and performs input/output operations.

(c) System Call Examples for Memory Allocation and Device Management

System calls allow programs to request services from the operating system.

1. Memory Management System Calls

- `mmap()` – Maps memory into a process's address space on Unix/Linux systems.
- `munmap()` – Releases a memory mapping.
- `brk()` / `sbrk()` – Adjust the process's data-segment boundary on systems that support them.

2. Device Management System Calls

- `open()` – Opens a device file for access.
- `read()` – Reads data from a device.
- `write()` – Sends data to a device.
- `ioctl()` – Performs device-specific control operations.
- `close()` – Closes the device file descriptor.

Program:

```
#include <stdio.h>

#include <sys/mman.h>

#include <fcntl.h>

#include <unistd.h>

int main()

{

   /* Memory allocation */

   void *ptr = mmap(NULL, 4096,

                    PROT_READ | PROT_WRITE,
```

```
                MAP_PRIVATE | MAP_ANONYMOUS,
                -1, 0);
 if (ptr == MAP_FAILED)
    {
       perror("Memory allocation failed");
       return 1;
    }
/* Device communication: write to a device file */
    int fd = open("/dev/null", O_WRONLY);
if (fd >= 0)
    {
       write(fd, "Hello", 5);
       close(fd);
    }
    /* Release allocated memory */
    munmap(ptr, 4096);
 return 0;
}
```

Explanation:

- `mmap()` allocates a memory mapping.
- `open()` opens the device file.
- `write()` sends data to the device.
- `close()` closes the device file.
- `munmap()` releases the memory mapping.